PART 7

Model Context Protocol

Five chapters on the standard that turns every integration into someone else's problem and every trust decision into yours: why the protocol exists, who the hosts and clients and servers are, how they connect and stay connected, what a server may offer beyond tools, and what it takes to ship one that survives an audit.

IN THIS PART

- 36. Why MCP Exists
- 37. Hosts, Clients and Servers
- 38. Transports and the Protocol Lifecycle
- 39. Resources, Tools and Prompts
- 40. Building, Securing and Shipping an MCP Server

CHAPTER 36

Why MCP Exists

In this chapter

Introduction: The Same Integration, Written Nine Times 4

36.1 Core Concepts 5

36.2 What MCP Standardises 6

36.3 The Connector, Concretely 7

36.4 Where the Connector Runs, and Where It Does Not 8

36.5 The Four Validation Rules, and the One That Is a Warning 9

36.6 Allowlist or Denylist: Ask About Tomorrow's Tool 9

36.7 What Part 6 Still Buys You 10

36.8 Analogies, and Where They Break 11

36.9 Architecture 11

36.10 Worked Example: Turning On One Server Responsibly 12

36.11 Code Walkthrough: An Audit That Reuses Five Chapters 13

36.12 Mini Project: Auditing Three Servers You Did Not Write 14

36.13 Audit One: Chapter 32's Coverage Becomes Zero 14

36.14 Audit Two: Names You Cannot Change, Words You Cannot Edit 15

36.15 Audit Three: The Error String Is Now Someone Else's 16

36.16 Audit Four: The Aggregate Catalogue 16

36.17 The Connector's Rules, Reproduced Offline 17

36.18 Hands-on Labs 17

36.19 Lab Solutions 18

36.20 Best Practices 20

36.21 Common Mistakes 20

36.22 Performance Considerations 22

36.23 Security Considerations 22

36.24 Real Production Examples 22

36.25 Certification Notes 23

36.26 Twenty Practice Questions 24

36.27 Answers 25

36.28 Ten Practical Exercises 26

36.29 Five Challenge Labs 26
36.30 Ten Interview Questions 26
36.31 Summary 27
36.32 Glossary 28
36.33 Further Reading 29

NOTE

Learning Objectives

By the end of this chapter you will be able to:

1. State the integration problem MCP exists to solve, and why it is multiplicative rather than additive.
2. Describe what MCP standardises and what it deliberately leaves to you.
3. Configure the **MCP connector** in the Messages API: `mcp_servers`, `mcp_toolset`, allowlists, denylists and per-tool configuration.
4. Reproduce the connector's documented validation rules, including the one that is a warning rather than an error, and say why that distinction matters.
5. Name what the connector supports of the MCP specification and what it does not, and choose between the connector and the client-side SDK helpers.
6. Audit a third-party server's tools with the machinery of Chapters 32 to 35, and say which of Part 6's guarantees survive.
7. Decide between an allowlist and a denylist on the evidence of what happens to a tool the server adds tomorrow.
8. Place the trust boundary correctly when the tool definitions, the descriptions and the error strings were written by someone else.

NOTE

Prerequisites

Chapter 26 for beta headers as dated, comma-separated names and for the OAuth surfaces. **Chapter 30** for the cached prefix and what invalidates it. **Chapter 31** for the tool-use contract and for tool results as untrusted input. **Chapter 32** for the declared-versus-enforced audit and what `strict: true` guarantees. **Chapter 33** for descriptions, namespacing and the linter. **Chapter 34** for the error contract, the four-criterion scorer, the leak scanner and call-site validation. **Chapter 35** for tool search, `defer_loading`, the aggregation arithmetic and the 30–50-tool accuracy threshold.

Deferred by name: **hosts, clients and servers** are **Chapter 37**; **transports and the protocol lifecycle** are **Chapter 38**; **resources, tools and prompts as protocol primitives** are **Chapter 39**; **building, securing and shipping a server** is **Chapter 40**. **MCP inside Claude Code** is **Chapter 41**; **prompt injection through a third-party tool result** is **Chapter 58**; **governance of an approved server list** is **Chapter 61**.

Introduction: The Same Integration, Written Nine Times

A platform team supports three assistants: a support agent, an engineering assistant and an analytics helper. Between them they need GitHub, Slack, Jira, Google Calendar, Snowflake and a home-grown deployment service. Six systems, three assistants.

The first year produced eighteen integrations. Not six — eighteen. Each assistant had its own tool definitions for each system, because each was written by a different team at a different time against a different framework. The GitHub tools existed three times with three descriptions, three schemas and three error strings. When GitHub added a field, three pull requests were needed and two were made.

This is the $N \times M$ problem, and it is the reason the Model Context Protocol exists. N applications and M systems produce $N \times M$ integrations if every pair is written separately, and $N + M$ if there is a protocol in the middle. Anyone who has watched an organisation write the same Jira wrapper for the fourth time understands the arithmetic without a diagram.

MCP is an open protocol that puts a standard interface between the two sides. A **server** exposes a system's capabilities. A **client** consumes them. Anyone can write either half, and the halves do not need to know about each other. Eighteen integrations becomes nine — six servers and three clients — and the second assistant to want Jira writes nothing at all.

That is the good news, and it is genuinely good. This chapter's job is to be equally clear about the invoice.

Part 6 spent five chapters teaching you to control a tool surface. Chapter 32 measured what your schema enforces — 16 constraints declared, 13 enforced, 81.2 per cent coverage. Chapter 33 argued that the description is the whole interface and gave you a linter for it. Chapter 34 built an error contract, a call-site validator and an idempotency discipline. Chapter 35 measured what a catalogue costs and when to defer it.

Now imagine all of that written by someone you have never met, shipped on their release schedule, and changed without notice.

§36.14 runs Chapter 32's audit over three plausible MCP servers — 55 tools between them. Their schemas declare **277 constraints**. Under `strict: true`, **211** would be enforced. Through the MCP connector, **zero** are, because the tool reference lists `strict` as available on *"All tools except* `mcp_toolset`*."* Chapter 32's 81.2 per cent becomes 0.0 per cent, and **66 of the unenforced constraints are the safety ones** — the minimums, maximums and lengths.

§36.15 finds that the 55 tools have only **52 distinct names**: three servers each expose a tool called `search`, and two expose `create_issue`. You cannot rename either. Chapter 33's linter reports **455 errors and 113 warnings** across descriptions you have no ability to edit. §36.16 scores the servers' own error strings with Chapter 34's scorer: **none of the three is actionable**, and one discloses an internal hostname straight into your conversation.

None of this is an argument against MCP. It is an argument for reading the chapter about it before turning it on. The single sentence to carry out of Part 7's opener is this: **MCP moves the integration work to someone else and leaves the trust boundary exactly where it was — at your call site.**

36.1 Core Concepts

DEFINITION

Model Context Protocol (MCP)

An open protocol for connecting AI applications to external systems through a standard interface, so that any compliant client can use any compliant server. The specification is versioned by date; the connector documentation links revision **2025-11-25**.

DEFINITION

MCP server

A process that exposes a system's capabilities over the protocol. It may be local — communicating over standard input and output — or remote, reachable over HTTP.

DEFINITION

MCP client

The component that speaks the protocol to a server on an application's behalf. Chapter 37 separates the client from the *host* that owns it, a distinction that matters more than it first appears.

DEFINITION

MCP connector

Anthropic's server-side MCP client, built into the Messages API. You name a remote server in `mcp_servers` and configure its tools with an `mcp_toolset` entry in `tools`; Anthropic connects to the server and runs the calls. Beta header `mcp-client-2025-11-20`.

DEFINITION

`mcp_toolset`

The entry in the `tools` array that configures one server's tools. It is a tool *type*, and it is the one type that accepts neither `strict` nor `allowed_callers`.

DEFINITION

Allowlist

A toolset with `default_config.enabled: false` and specific tools re-enabled in `configs`. A tool the server adds tomorrow is **disabled**.

DEFINITION

Denylist

A toolset with defaults left alone and specific tools disabled in `configs`. A tool the server adds tomorrow is **enabled**.

DEFINITION

Transport

How client and server exchange messages. The connector requires a publicly reachable HTTPS server and supports **Streamable HTTP** and **SSE**; **local STDIO servers cannot be connected directly**. Chapter 38 takes transports properly.

36.2 What MCP Standardises

The protocol's value is not that it is clever; it is that it is *agreed*. Three things become uniform.

Discovery. A client asks a server what it offers and receives a machine-readable answer. Your application no longer hard-codes a tool list; it reads one.

Invocation. A tool call has one shape regardless of which server implements it, so a client written once works against a server written later.

Capability negotiation. Client and server agree at connection time on what each supports, so a newer server and an older client can still talk. This is the same instinct as Chapter 26's `anthropic-version` contract, arriving in a different protocol.

The MCP specification defines more than tools — resources and prompts on the server side, and features such as sampling and elicitation on the client side. **The Claude connector supports only tool calls.** The documentation is explicit: *"Of the feature set of the MCP specification, only tool calls are currently supported."* Chapter 39 covers the other primitives and the client-side SDK helpers that reach them.

What MCP does **not** standardise is as important. It says nothing about authorisation policy, about rate limits, about idempotency, about what an error string should contain, or about whether a tool is safe to call twice. Those were your problems in Part 6 and they are still your problems. The protocol standardises the *plumbing*; every argument in Chapters 32 to 35 was about the *water*.

36.3 The Connector, Concretely

Two components, and keeping them straight is most of the learning curve. `mcp_servers` says **where** and **who**; the `mcp_toolset` entry in `tools` says **which** and **how**.

```
{
  "model": "claude-opus-5",
  "max_tokens": 1000,
  "messages": [{"role": "user", "content": "What's blocking the release?"}],
  "mcp_servers": [
    { "type": "url",
      "url": "https://example-server.modelcontextprotocol.io/sse",
      "name": "example-mcp",
      "authorization_token": "YOUR_TOKEN" }
  ],
  "tools": [
    { "type": "mcp_toolset", "mcp_server_name": "example-mcp" }
  ]
}
```

Server fields: `type` (only `"url"` is supported), `url` (must start `https://`), `name` (a unique identifier referenced by **exactly one** toolset), and an optional `authorization_token` carrying an OAuth bearer that **you** obtain and refresh — the API does not run the OAuth flow for you.

Toolset fields: `mcp_server_name`, an optional `default_config`, an optional `configs` map keyed by tool name, and an optional `cache_control` breakpoint. Each tool configuration supports `enabled` (default `true`) and `defer_loading` (default `false`).

Configuration merges with a precedence you should be able to recite: `configs` **beats** `default_config` **beats the system defaults.** The documented example is worth memorising because it is the shape of every real configuration:

```
{ "type": "mcp_toolset", "mcp_server_name": "google-calendar-mcp",
  "default_config": { "defer_loading": true },
  "configs": { "search_events": { "enabled": false } } }
```

`search_events` ends up `enabled: false` from `configs` and `defer_loading: true` from `default_config`; every other tool is `enabled: true` from the system default and `defer_loading: true` from `default_config`. §36.18 reproduces both resolutions in a test.

Two new content blocks

```
{ "type": "mcp_tool_use", "id": "mcptoolu_014Q35Rayj...",
  "name": "echo", "server_name": "example-mcp",
  "input": { "param1": "value1" } }
{ "type": "mcp_tool_result", "tool_use_id": "mcptoolu_014Q35Rayj...",
  "is_error": false,
  "content": [ { "type": "text", "text": "Hello" } ] }
```

Note `server_name` on the call — with several servers connected, it is the only thing telling you whose `search` was invoked — and note that the *result* carries `is_error`, filled in by the API from the server's response. Chapter 34's whole discipline about error strings applies, and none of the levers do.

36.4 Where the Connector Runs, and Where It Does Not

Four constraints decide whether the connector is even an option, and all four were verified on 4 August 2026.

Constraint	Value
Beta header	`mcp-client-2025-11-20`; `mcp-client-2025-04-04` is **deprecated**
MCP features supported	**Tool calls only**
Transport	Public HTTPS, Streamable HTTP or SSE. **Local STDIO cannot be connected directly**
Platforms	Claude API, Claude Platform on AWS, Microsoft Foundry (**Hosted on Anthropic deployments only**). **Not Amazon Bedrock, not Google Cloud**

Two more that catch people out. The connector **is not covered by zero data retention**: *"Data exchanged with MCP servers, including tool definitions and execution results, is retained according to Anthropic's standard data retention policy."* And it **is** usable in the Message Batches API, priced the same as in a synchronous request.

The deprecated beta header is worth recognising because you will meet it in old code. In `mcp-client-2025-04-04`, tool configuration lived **inside** the server definition as `tool_configuration` with an `allowed_tools` array. In the current version it lives in `tools` as an `mcp_toolset`. The migration is mechanical: `tool_configuration.enabled: false` becomes `default_config.enabled: false`, and

`allowed_tools: [...]` becomes `default_config.enabled: false` plus per-tool `enabled: true` in `configs`.

> TIP
>
> **When to use the client-side helpers instead**
>
> The documentation draws the line cleanly. Use `mcp_servers` *"when you have remote servers accessible by URL and only need tool support."* Use the SDK's client-side helpers *"when you need local servers, prompts, resources, or more control over the connection."* Those helpers — `mcpTools`, `mcpMessages`, `mcpResourceToContent`, `mcpResourceToFile` in their TypeScript forms — convert between MCP types and Claude API types, and they throw an `UnsupportedMCPValueError` when an MCP value has no Claude equivalent. Chapter 39 uses them.

36.5 The Four Validation Rules, and the One That Is a Warning

The connector enforces four rules. Three raise; one does not, and the one that does not is the one with security consequences.

1. **The server must exist.** An `mcp_server_name` that matches no entry in `mcp_servers` is an error.
2. **The server must be used.** Every server defined must be referenced by exactly one toolset.
3. **One toolset per server.** A server referenced twice is an error.
4. **Unknown tool names in `configs`** — *"a backend warning is logged but no error is returned (MCP servers may have dynamic tool availability)."*

Read the fourth one twice. If you write a denylist and misspell the name of the destructive tool, **nothing tells you**. The request succeeds, the tool stays enabled, and your protection exists only in the configuration file. The documented reason is sound — MCP servers can change their tool list at run time, so a strict check would break legitimate configurations — but the consequence is yours to manage.

> COMMON MISTAKE
>
> **A denylist with a typo**
>
> **Symptom.** A tool you believed was disabled gets called.
>
> **Cause.** `configs` keys are tool names on a server you do not control, and an unmatched key is a backend warning rather than a 400.
>
> **Fix.** Prefer an allowlist for anything that can change state. Where a denylist is unavoidable, fetch the server's tool list yourself and assert that every key in `configs` exists, before the request is sent.
>
> **Detection.** §36.18's `validate()` returns the warning list rather than swallowing it; fail your build on a non-empty list.

36.6 Allowlist or Denylist: Ask About Tomorrow's Tool

Both patterns are documented, and the documentation recommends denylisting write or destructive tools *"when building read-only assistants, or when you want a human confirmation step before state changes."* That is good advice about which tools to name. It is not advice about which pattern to choose,

and the two differ on exactly one question.

What happens to a tool the server adds next week?

Under a **denylist**, it is enabled, because `enabled` defaults to `true`. Under an **allowlist**, it is disabled, because your `default_config` says so.

§36.18 makes the difference a number. On a 32-tool server, a denylist of two leaves **30 enabled**; an allowlist of three leaves **3**. Add one tool to the server and the denylist leaves **31** while the allowlist still leaves **3**. The server's release notes are now part of your security posture under one pattern and not the other.

The audit in §36.15 finds **10 of the 55 tools** across three servers whose names alone suggest a state change — `create_issue`, `close_issue`, `merge_pr`, `delete_branch`, `create_event`, `delete_all_events`, `share_calendar_publicly`, `create_issue` again, `transition`, `refund_order`. A denylist has to name all ten and stay correct as the servers change. An allowlist has to name none of them and stays correct by construction.

KEY TAKEAWAYS

The rule

Choose an allowlist unless you can explain what will happen to a tool that does not exist yet.

36.7 What Part 6 Still Buys You

Take the five chapters in order and ask what survives.

Chapter 31 — the loop. Survives unchanged in principle and changes in practice: with the connector, *Anthropic's* loop calls the tool, and what you receive is an `mcp_tool_result` that has already happened. The blocks are different, the contract is the same, and the rule that the model never executes anything is still true — it is simply someone else's process doing the executing.

Chapter 32 — the schema. Does not survive. `strict` is documented as available on *"All tools except* `mcp_toolset`." You cannot turn constrained decoding on for connector tools, so the 277 declared constraints in §36.14 are enforced by nothing. Chapter 32's supported-subset arithmetic still tells you what *would* be enforced, and that number — 211 — is now a description of what you are missing.

Chapter 33 — the words. Survives as a diagnostic and not as a lever. You can lint a third-party description; you cannot fix it. What you *can* do is disable the tool, or add a disambiguating sentence to your system prompt, which is why §36.15's collision count is actionable even though the names are not.

Chapter 34 — the error contract. Survives at the call site and nowhere else. The `is_error` string comes from the server. §36.16 scores three of them at 0 for 3 actionable, with one leaking an internal hostname. Your options are to disable the tool, to wrap the server (which means running your own MCP client rather than the connector), or to accept it.

Chapter 35 — the catalogue. Survives entirely, and is the most useful of the five here. `defer_loading` works on MCP toolsets — set on `default_config` for the whole server or per tool in `configs` — and the aggregation is exactly what tool search was built for. §36.17 puts the three servers at **55 tools and**

25,300 definition tokens, above the documented 30-to-50 accuracy threshold and past Chapter 35's computed crossover.

```
YOUR OWN TOOLS (Part 6)              MCP CONNECTOR TOOLS (Part 7)
─────────────────────────            ──────────────────────────────
name              you                name              THE SERVER
description       you                description       THE SERVER
input_schema      you                input_schema      THE SERVER
strict: true      you                strict            NOT AVAILABLE
input_examples    you                input_examples    NOT AVAILABLE
allowed_callers   you                allowed_callers   NOT AVAILABLE
the executor      you                the executor      ANTHROPIC
the error string  you                the error string  THE SERVER
──────────────────────────────────────────────────────────────────
enabled / disabled              YOU  (mcp_toolset configs)
defer_loading                   YOU  (default_config or configs)
the system prompt around it     YOU
cache_control on the toolset    YOU
WHAT YOUR APPLICATION DOES      YOU  <- the only line that matters
WITH THE RESULT
```

The last line is the chapter's thesis. Every gate Chapter 34 built lives *after* the tool result arrives, and a result you did not produce is exactly the untrusted input Chapter 31 warned about — only now it is untrusted by construction rather than by accident.

36.8 Analogies, and Where They Break

VISUAL ANALOGY

USB before and after

Before a standard connector, every peripheral shipped its own port, cable and driver, and every computer decided which peripherals it would support. USB replaced $N \times M$ with $N + M$: any device, any host, one shape.

Where it breaks. A USB device does what it says on the box, and a rogue one is a well-known hardware attack for exactly this reason. An MCP server hands your model *text*, and text is instructions to a component that reads instructions. The analogy is right about the economics and understates the trust question by a wide margin.

VISUAL ANALOGY

Hiring a contractor with their own tools

You bring in a specialist and they arrive with a full toolbox. That is efficient and it is why you hired them. You still decide which jobs they are allowed to do, you still check the work before it goes live, and you do not hand them the keys to the building because their van is well organised.

Where it breaks. A contractor's toolbox does not gain a new power tool overnight without telling you, which is precisely what a denylisted MCP server can do. And a contractor understands your intent; a server exposes capability with no view about your policy at all.

36.9 Architecture

```
  (a) MCP CONNECTOR - Anthropic runs the client
      your app ──POST /v1/messages──► Anthropic ──MCP/HTTPS──► server
                mcp_servers + mcp_toolset          (public, https)
      you see: mcp_tool_use / mcp_tool_result, already executed
      you get: tools only. no strict, no allowed_callers, no ZDR
      you keep: enabled, defer_loading, cache_control, the prompt
  (b) CLIENT-SIDE SDK HELPERS - you run the client
      your app ──MCP──► server (stdio OR https, local OR remote)
          │  mcpTools() / mcpMessages() / mcpResourceToContent()
          └──POST /v1/messages──► Anthropic   (ordinary tools array)
      you see: ordinary tool_use blocks; YOU execute
      you get: resources and prompts as well as tools
      you keep: EVERYTHING in Part 6, because you own the wrapper
  (c) MCP TUNNELS  (Ch 26: beta mcp-tunnels-2026-06-22,
      workspace:manage_tunnels)  -- a private server reachable by
      Anthropic without being public. Chapter 38 takes this properly.
```

The choice between (a) and (b) is the choice between convenience and control, and it is not a close call for anything that changes state. Route (b) is more code and it puts every gate from Chapter 34 back in your hands: you validate at the call site, you mint the idempotency key, you write the error string, you authorise the caller. Route (a) is one JSON object and gives you `enabled`, `defer_loading` and a system prompt.

A defensible default for most organisations: **route (a) for read-only servers, route (b) for anything with an effect.**

36.10 Worked Example: Turning On One Server Responsibly

A team wants the calendar server. Six steps, and only the first is about MCP.

One — read the manifest before you connect it. Fetch `tools/list` from the server and look at it. §36.13's audit is exactly this, automated.

Two — count the effects. The calendar server exposes fourteen tools, of which `create_event`, `delete_all_events` and `share_calendar_publicly` change state. `delete_all_events` takes a single `confirm` string, which is a confirmation the model can write.

Three — choose the pattern. The assistant needs to read the calendar and create events. That is an allowlist of two, not a denylist of three.

```
{ "type": "mcp_toolset", "mcp_server_name": "calendar-mcp",
  "default_config": { "enabled": false, "defer_loading": true },
  "configs": { "list_events": { "enabled": true,
                                "defer_loading": false },
               "create_event": { "enabled": true } } }
```

Four — put the effect behind your own gate. `create_event` is enabled here, which means Anthropic's loop will call it and you will never see the request. If that is unacceptable — and for a shared calendar it usually is — the tool belongs on route (b), wrapped in your own tool with Chapter 34's five gates in front of it. There is no middle setting.

Five — disambiguate the name. The calendar server's `search` collides with two other servers' `search`. One sentence in the system prompt naming the categories is the documented remedy and the cheapest one: *"Use calendar tools for events and availability; use ticket tools for customer issues."*

Six — write down what you saw. The server's tool list, the date you read it, and the ten constraints its schemas declare that nothing enforces. That record is what makes the next upgrade a review rather than a surprise.

36.11 Code Walkthrough: An Audit That Reuses Five Chapters

The mini project contains almost no new logic. That is the point: the machinery of Part 6 turns out to work perfectly well on tools you did not write, and what changes is the column it fills in.

```
def constraint_audit(tools=None):
    """Declared across the estate; enforced by the API: zero."""
    tools = tools or [t for _s, t in C.all_tools()]
    declared = enforceable = safety = 0
    for t in tools:
        a = E.audit(t.input_schema)
        declared += len(a["declared"])
        enforceable += len(a["enforced"])
        safety += sum(1 for _p, k in a["stripped_to_description"]
                      + a["dropped"] if k in ("minimum", "maximum",
                                              "maxLength", "minLength",
                                              "maxItems"))
    return {"tools": len(tools), "declared": declared,
            "enforceable_if_strict": enforceable,
            "enforced_via_mcp_toolset": 0,
            "safety_constraints_unenforced": safety}
```

`E.audit` is Chapter 32's function, unmodified. It reports what the grammar would enforce **if a grammar existed**, which is why the return value keeps `enforceable_if_strict` and `enforced_via_mcp_toolset` as separate keys rather than collapsing them. The second is a hard-coded zero, and hard-coding it is deliberate: it is a documented fact — `strict` is available on *"All tools except* `mcp_toolset`*"* — and a constant that a reader can grep for is better than a subtraction they have to trust.

`safety` counts only the keywords Chapter 32 identified as the dangerous misses. Reporting it separately is the difference between "some constraints are unenforced" and "the range checks are unenforced".

The second walkthrough is the validator, and the interesting part is what it returns rather than what it raises.

```
    warnings = []
    known = known_tools or {}
    for t in sets:
        for tool_name in (t.get("configs") or {}):
            avail = known.get(t["mcp_server_name"])
            if avail is not None and tool_name not in avail:
                warnings.append(
                    "configs names %r, which does not exist on %r; a "
                    "backend warning is logged and NO ERROR is "
                    "returned ..." % (tool_name, t["mcp_server_name"]))
    return warnings
```

Three rules raise; this one appends. `validate` returns the warning list rather than logging it, because a caller that never looks at the return value has made a choice and a caller that logs it has not. The `avail is not None` guard matters too: if you have not fetched the server's tool list, the check is skipped rather than guessed, which is the same UNKNOWN discipline Chapter 14 established — a fact you have not verified does not get a default.

36.12 Mini Project: Auditing Three Servers You Did Not Write

What it is. Three BUILT MCP manifests — a code-hosting server with 32 tools, a calendar server with 14, a ticketing server with 9 — put through Chapters 32, 33, 34 and 35's machinery, plus an offline reproduction of the connector's request shape and validation rules.

```
ch36/
├── enforce.py     ─┐ ch32: the declared-vs-enforced audit
├── schema.py       │ ch23/ch32: the supported subset
├── tooldef.py      │ ch32: the Tool object
├── callsite.py    ─┘ ch34: the validator derived from the audit
├── lintdesc.py    ─┐ ch33: the documented description rules
├── affordance.py  ─┘ ch33: tokenisation for the OVERLAP rule
├── errmsg.py         ch34: four-criterion scorer, leak scanner
├── errcorpus.py      ch34: the 40 BUILT failures
├── toolsearch.py     ch35: defer_loading economics, thresholds
├── costmodel.py      ch11: the verified rate card
├── cacheplan.py      ch30: breakpoints and multipliers
│
├── mcpcat.py         NEW  three manifests, 55 tools, BUILT
├── mcpaudit.py       NEW  five audits, no new logic
├── mcpconn.py        NEW  the connector shape + 4 rules, offline
├── report.py         NEW  every published figure
└── test_ch36.py      NEW  24 tests
```

```
cd ch36
pip install --break-system-packages pytest
python3 -m pytest -q      # 24 passed
python3 report.py
```

Provenance. Everything in `mcpconn.py` is **DOC**, transcribed from the MCP connector page on 4 August 2026: the beta headers, the field tables, the merge precedence, the four validation rules, the two content block types, the platform list, the ZDR statement and the transport constraints. The three manifests, their descriptions, their schemas and their error strings are **BUILT** — constructed to look like plausible third-party output. No request was made to any server.

36.13 Audit One: Chapter 32's Coverage Becomes Zero

```
1. Chapter 32's audit, on tools you did not write
--------------------------------------------------------------------
tools across three servers            55
constraints their schemas declare     277
enforceable if strict were available  211
actually enforced via mcp_toolset     0
of which safety constraints unenforced 66
```

Two hundred and seventy-seven constraints, and the API enforces none of them, because `strict` is not available on an `mcp_toolset`. Chapter 32's payments tool had 81.2 per cent coverage and the chapter called the three misses alarming because they were the range checks. Here the coverage is **0.0 per cent** and **66** of the misses are range checks.

Read that carefully, because it is easy to over-read. The server presumably validates its own inputs — a competent Jira server will reject a priority outside its enum whatever Claude sends. What is gone is the *pre-execution* guarantee: with `strict: true` on your own tool, a malformed call cannot be generated. Without it, the call is generated, transmitted, executed against a third party, and rejected by them — which is a round trip, an audit-log entry on their side, and, for anything with an effect, a partial action you now have to reason about.

The remedy is the one Chapter 34 already built and it is unchanged: **validate at the call site.** The only difference is that for connector tools there is no call site of yours to put it at, which is the sharpest argument in this chapter for wrapping effect-carrying MCP tools in your own instead.

36.14 Audit Two: Names You Cannot Change, Words You Cannot Edit

```
2. Chapter 33's namespacing argument, as a collision count
------------------------------------------------------------------------
tools 55, distinct names 52
  create_issue   exposed by code-mcp, tickets-mcp
  search         exposed by code-mcp, calendar-mcp, tickets-mcp
  Chapter 33's linter, on descriptions you cannot edit:
  server          tools   errors   warns
  code-mcp           32      364      65
  calendar-mcp       14       64      29
  tickets-mcp         9       27      19
  TOTAL                      455     113
```

Chapter 33 said namespacing is *"especially important when using tool search"* and made the argument in the abstract. Here it is a count: three servers, three tools called `search`, and a model asked *"search for the release blocker"* has to pick between a code search, a calendar search and a ticket search on descriptions of four, three and three words respectively.

Chapter 33 measured what an undifferentiated catalogue does — 34 of 40 requests ending in a **tie**, an absent preference resolved by an arbitrary rule. That measurement was a proxy and is quoted here as one, but the mechanism is not in doubt and the vocabulary here is thinner than Chapter 33's "before" catalogue.

The 455 linter errors are worth taking with some salt and are worth reporting anyway. Chapter 33's rules are Anthropic's documented rules for tool descriptions, and third-party MCP servers were not written against them — most descriptions are a single sentence, few say when *not* to use the tool, and almost none say what the tool does not return. That is not negligence; it is what the ecosystem looks like. What the count tells you is how much disambiguation your system prompt is being asked to do.

Three levers remain, and all three are yours:

- **Disable the duplicates.** If the assistant never needs code search, `enabled: false` removes the collision entirely. This is the cheapest fix and the most under-used.
- **Name the categories in the system prompt.** The tool search page recommends exactly this: *"You can search for tools to interact with Slack, GitHub, and Jira."*
- **Defer the lot and let search decide.** Which moves the problem to retrieval quality, and Chapter 35 measured what that costs.

36.15 Audit Three: The Error String Is Now Someone Else's

```
3. Chapter 34's scorer, on the server's error strings
----------------------------------------------------------------------
  code-mcp       actionable=False leaks=-
                 'Error: 422 Unprocessable Entity'
  calendar-mcp   actionable=False leaks=internal_host
                 'RuntimeError: calendar backend unavailable at
                  cal-api.internal.example:8443'
  tickets-mcp    actionable=False leaks=-
                 'failed'
```

Three servers, three arms of Chapter 34's experiment, arriving without being asked for. `'failed'` is the terse arm — the string the documentation names as the thing not to write. The 422 is terse with a status code, which names neither the field nor the constraint. The calendar server is the raw-exception arm, and it puts an internal hostname into your conversation, where it is re-sent as input on every subsequent turn.

None of the three is actionable on Chapter 34's four criteria. That matters more than it did in Chapter 34, because the documented recovery budget has not changed: an invalid tool call is retried **two to three times** before Claude apologises to the user, and a string that names nothing spends all three on guesses.

You cannot fix any of it. The string arrives inside an `mcp_tool_result` block that the API constructs from the server's response, and by the time you see it the call has happened. Three things you *can* do:

1. **Disable the tool** if its failures are unusable.
2. **Wrap the server** — run your own MCP client, catch the result, and return Chapter 34's instructive string yourself. This is route (b) of Figure 36.2 and it is the only complete answer.
3. **Scan, and report upstream.** `errmsg.scan` over `mcp_tool_result` content in your logs will find the disclosures, and a well-run server will accept the bug report. Chapter 40 is about being on the other end of that report.

> WARNING
>
> **The disclosure is not yours and the consequence is**
>
> An internal hostname from a third-party server is in *your* conversation, *your* context window, potentially *your* cache entry, and potentially quoted to *your* user. "The server did it" is a true statement and not a control.

36.16 Audit Four: The Aggregate Catalogue

```
4. Chapter 35's arithmetic, on the aggregate catalogue
----------------------------------------------------------------------
tools 55, definition tokens 25300
above the documented 30-50 accuracy threshold: True
cached baseline 2530 units/turn, deferred 1626 (36% less)
```

Three servers is not many, and three servers is already past the point where the documentation says selection accuracy degrades. The tool search page names the aggregation case explicitly as a reason to adopt search: *"You aggregate multiple MCP servers (200+ tools)."* Fifty-five is not two hundred, and it is over the threshold.

At 25,300 definition tokens the catalogue is past Chapter 35's computed crossover of 16,260 under prefix-only caching, so deferring is cheaper as well as more accurate — **36 per cent fewer units per steady-state turn**. Note that the saving is smaller than Chapter 35's 71 per cent at the five-server scale, which is what you would expect: the crossover is a curve, not a switch, and 55 tools sits nearer the knee.

One configuration detail is specific to MCP and easy to miss. You do **not** set `defer_loading` on individual definitions, because you did not write them. It goes on the toolset's `default_config` for the whole server, or per tool in `configs`:

```
{ "type": "mcp_toolset", "mcp_server_name": "code-mcp",
  "default_config": { "defer_loading": true },
  "configs": { "search": { "defer_loading": false } } }
```

36.17 The Connector's Rules, Reproduced Offline

```
5. The connector's request shape and its four rules
----------------------------------------------------------------------
    a valid single-server request              accepted
    http:// instead of https://                rejected
    type other than "url"                      rejected
    toolset names a server that is not defined rejected
    a server defined but never referenced      rejected
    two toolsets for one server                rejected
  5 rejections and 1 acceptance reproduced offline
  a configs entry naming a tool the server does not have:
    warnings=1, exception=none
  code-mcp has 32 tools.
    denylist of 2 leaves 30 enabled
    allowlist of 3 leaves 3 enabled
    tools whose NAME suggests an effect: 4
```

Five rejections and one acceptance, with no key and no server. The sixth case is the important one and it is not in the list because it does not raise: a `configs` entry naming a tool the server does not have returns a **warning** and the request succeeds.

The allowlist arithmetic is the practical output of the whole chapter. Thirty enabled against three. And when the server ships a new tool, `test_allowlist_and_denylist_differ_on_tomorrows_tool` asserts what happens: the denylist goes to 31 and the allowlist stays at 3.

36.18 Hands-on Labs

INDUSTRY USE CASE

Lab 1 — Read a real manifest

Pick an MCP server you are considering. Fetch its tool list and count: total tools, distinct names, tools whose names suggest an effect, and total description characters.

Success criterion. Four numbers, and a sentence about whether the catalogue is above the 30–50 threshold on its own.

INDUSTRY USE CASE

Lab 2 — Run the audit on it

Convert the manifest into `tooldef.Tool` objects and run `mcpaudit.constraint_audit` and `mcpaudit.lint` over it.

Success criterion. You can state how many constraints the server declares that nothing will enforce, and how many of them are range checks.

INDUSTRY USE CASE

Lab 3 — Write the allowlist

Produce the `mcp_toolset` configuration your assistant actually needs, then compute how many tools it leaves enabled against a denylist covering the same effect tools.

Success criterion. Two counts, and a decision recorded with a reason.

INDUSTRY USE CASE

Lab 4 — Break the denylist on purpose

Write a denylist with a deliberately misspelled tool name and confirm that the request is accepted.

Success criterion. You have seen the silent failure yourself, and you have added an assertion to your build that would catch it.

INDUSTRY USE CASE

Lab 5 — Wrap one effect tool

Take one state-changing MCP tool and move it to route (b): your own MCP client, your own wrapper tool, Chapter 34's five gates in front of it.

Success criterion. The wrapper validates at the call site, mints an idempotency key, authorises the caller and returns an instructive error string. Score the string with `errmsg`.

36.19 Lab Solutions

Lab 1. Four counts, and the third is the one that changes your configuration:

```
names = [t["name"] for t in manifest["tools"]]
effects = [n for n in names
           if any(c in n for c in mcpcat.EFFECT_CUES)]
print(len(names), len(set(names)), len(effects),
      sum(len(t.get("description", "")) for t in manifest["tools"]))
```

The name-based effect scan is a first pass, not an answer. A tool called `transition` changes state and a tool called `get_report` might trigger a billing event. Read the server's documentation for anything you intend to enable.

Lab 2. The conversion is mechanical and the audit is unchanged:

```
tools = [Tool(t["name"], t.get("description", ""),
              t["inputSchema"]) for t in manifest["tools"]]
print(mcpaudit.constraint_audit(tools))
print(lintdesc.summary(lintdesc.lint_catalogue(tools)))
```

MCP manifests use `inputSchema` where the Claude API uses `input_schema`; that rename is the only translation this needs, and it is a small preview of Chapter 39.

Lab 3. Compute both, then choose:

```
allow = mcpconn.allowlist("srv", needed)
deny = mcpconn.denylist("srv", effects)
print(mcpconn.enabled_count(allow, names),
      mcpconn.enabled_count(deny, names))
```

If the two counts are close, the server is small and either pattern works. If they are far apart — 3 against 30 in §36.17 — the allowlist is describing your intent and the denylist is describing the server's.

Lab 4. One character is enough:

```
ts = mcpconn.denylist("code-mcp", ["delete_branchh"])
print(mcpconn.validate(SRV, [ts], {"code-mcp": names}))
```

The warning list is non-empty and nothing raised. Put that assertion in your build:

```
warnings = mcpconn.validate(servers, tools, fetched_tool_names)
assert not warnings, warnings
```

Lab 5. The wrapper is ordinary Part 6 code, and its shape is the deliverable:

```
def calendar_create_event(principal, tool_input, logical_id):
    authz.authorise(principal, "calendar_create_event")
    bad = validator.check(tool_input)
    if bad:
        return errmsg.instructive(bad[0], expectation(bad[0]))
    key = "loop:%s" % logical_id
    if not ledger.apply(key, now()):
        return "Already created for this request."
    return mcp_client.call_tool("create_event", tool_input)
```

Five gates, one call to the server, and every string the model reads written by you. Compare it with the connector's one JSON object and decide per tool, not per server.

36.20 Best Practices

BEST PRACTICE

The twelve rules

1. **Read the manifest before you connect the server,** and record what you read with a date.
2. **Prefer an allowlist** for anything that can change state; a denylist inherits the server's release notes.
3. **Assert that every `configs` key exists** on the server, because the API will not.
4. **Route read-only servers through the connector and effect-carrying tools through your own client.** There is no middle setting.
5. **Never assume a schema constraint is enforced.** `strict` is unavailable on `mcp_toolset`; 277 declared constraints bought nothing in §36.13.
6. **Scan `mcp_tool_result` content with a leak scanner** and treat a hit as a bug to report upstream and a tool to disable meanwhile.
7. **Disambiguate colliding names in the system prompt,** or disable one side of the collision.
8. **Set `defer_loading` on `default_config`,** not on individual definitions you did not write.
9. **Put the toolset's `cache_control` breakpoint deliberately;** the tools array is the first cache prefix level (Chapter 30).
10. **Remember the connector is not ZDR-eligible,** and check that against your data-handling commitments before connecting anything sensitive.
11. **Obtain and refresh OAuth tokens yourself;** the connector takes a bearer, it does not run the flow.
12. **Re-audit on every server upgrade.** The manifest is a dependency, and it has no lockfile.

36.21 Common Mistakes

COMMON MISTAKE

Treating an MCP server as trusted because it is official

Symptom. A well-known vendor's server is connected with no allowlist and no review.

Cause. Confusing the *provenance* of the code with the *content* of what it returns. Chapter 31's rule is about the content: a tool result is untrusted because of where its data comes from, not because of who wrote the server.

Fix. Allowlist regardless of vendor. Review the effect tools regardless of vendor.

Detection. Ask what the server would return if the underlying system contained an attacker-authored record. Chapter 58 is the full treatment.

COMMON MISTAKE

Assuming `strict: true` applies

Symptom. Malformed calls reach a third-party API and are rejected there.

Cause. `strict` is documented as available on *"All tools except* `mcp_toolset`*"*.

Fix. Wrap effect tools, or accept the round trip on read-only ones.

Detection. §36.13: 277 declared, 0 enforced.

COMMON MISTAKE

Connecting a server on Bedrock or Google Cloud

Symptom. The request is rejected on a platform where the rest of your integration works.

Cause. The MCP connector is available on the Claude API, Claude Platform on AWS and Microsoft Foundry (Hosted on Anthropic only). It is not available on Amazon Bedrock or Google Cloud.

Fix. Route (b): run the MCP client yourself and send ordinary tools.

Detection. A platform matrix in your registry, in the shape Chapter 14 built.

COMMON MISTAKE

Expecting resources and prompts through the connector

Symptom. A server's documents or prompt templates are invisible.

Cause. *"Of the feature set of the MCP specification, only tool calls are currently supported."*

Fix. The client-side SDK helpers — `mcpResourceToContent`, `mcpResourceToFile`, `mcpMessages` — reach them. Chapter 39 uses all three.

COMMON MISTAKE

Pointing the connector at a local server

Symptom. A server that works in your editor cannot be connected.

Cause. The connector needs a publicly reachable HTTPS endpoint; local STDIO servers cannot be connected directly.

Fix. The client-side helpers for local servers, or MCP tunnels for a private server that Anthropic must reach. Chapter 38 covers both.

COMMON MISTAKE

Leaving the deprecated beta header in place

Symptom. `tool_configuration` and `allowed_tools` in a working request, and no access to per-tool configuration.

Cause. `mcp-client-2025-04-04` is deprecated in favour of `mcp-client-2025-11-20`.

Fix. Move the configuration into an `mcp_toolset`: `enabled: false` becomes `default_config.enabled: false`, and `allowed_tools` becomes per-tool `enabled: true` in `configs`.

Detection. Grep for the old header; Chapter 26 established that the date is part of the name.

36.22 Performance Considerations

Definitions dominate. Three modest servers is 25,300 definition tokens in the cached prefix. Chapter 30's arithmetic applies unchanged: put the `cache_control` breakpoint on the toolset, and remember that the tools array is the *first* prefix level, so any change to a server's manifest invalidates everything after it.

A server upgrade is a cache invalidation. This is the one performance consequence unique to MCP. Your prefix now contains text the server controls, and the day they add a tool your cache hit rate goes to zero without any change on your side. `defer_loading: true` on the toolset largely removes this, because deferred tools are excluded from the prefix before the cache key is computed.

Latency has an extra hop. Anthropic's client connects to the server's public endpoint, so a slow server is slow inside your turn rather than inside your process, and you cannot instrument it. Route (b) puts the hop back in your process where you can time it.

Batch works and is priced the same. MCP tool calls through the Batches API cost what they cost synchronously, and Chapter 30's 50 per cent discount applies to the tokens as usual.

36.23 Security Considerations

The trust boundary did not move. Everything in Chapter 34's Figure 34.1 still applies; what changed is that for connector tools, gates 1 to 4 have no place to run. That is the security argument for route (b), stated as plainly as it can be.

Tool descriptions are inputs to your model. A server's description text enters your system prompt prefix at the highest-authority position a tool has (Chapter 33). A malicious or compromised server can therefore write into that channel. Reviewing descriptions before enabling a server is not pedantry; it is reading the code you are about to run.

Dynamic tool availability cuts both ways. The documented reason `configs` does not validate is that servers can change their tool list. The same property means a server can add a tool after your review. Allowlist.

Data leaves your boundary in both directions. Tool inputs go to the server; results come back into your context. The connector is **not ZDR-eligible**, and the documentation says so explicitly for both tool definitions and execution results.

Tokens are long-lived and yours to rotate. `authorization_token` is an OAuth bearer you obtained and must refresh. Chapter 26's rules about credentials apply: it belongs in a secret store, it does not belong in a prompt, and it should be scoped as narrowly as the server permits.

MCP tools cannot be called programmatically. The programmatic tool calling page lists MCP connector tools among the tools that cannot currently be called from within code execution. That closes one route by which a script could invoke them in bulk, and it is a limitation to plan around rather than a control to rely on.

36.24 Real Production Examples

A read-only research assistant. Four MCP servers, all connector-mounted, every toolset `default_config.enabled: false` with six tools allowlisted in total. No effect tools enabled anywhere. The team's rule is one sentence: if a tool can change something, it does not go through the connector.

A support agent with one wrapped write. Ticket search and customer lookup through the connector; the refund tool wrapped in the team's own MCP client with Chapter 34's five gates. The wrapper is forty lines and is the only place their auditors look.

An aggregation that hit the threshold. Five servers, 180 tools, `defer_loading: true` on every `default_config` and the tool search tool alongside. Selection accuracy was the reason; the token saving was the argument that got it approved.

A server upgrade that invalidated a cache. A vendor added eleven tools on a Tuesday. Cache hit rate fell to zero, the input bill tripled for a day, and nobody had changed anything. The fix was deferral, which had been on the backlog as a cost item and turned out to be a stability item.

A denylist that missed. A misspelled key in `configs`, no error, and a destructive tool enabled for three weeks. The remediation was the assertion in §36.19's Lab 4 and a move to allowlists across the estate.

36.25 Certification Notes

Concept	Where it is examined	Domain
The $N \times M$ problem and what MCP standardises	MCP fundamentals	D2 Tool Design & MCP Integration
`mcp_servers` and `mcp_toolset` request shape	Connector configuration	D2
Config merge precedence and the four validation rules	Connector configuration	D2
Allowlist versus denylist and future tools	Least privilege	D2, D5
Connector limitations: tools only, HTTPS, platforms, ZDR	Deployment constraints	D2, D5
`strict` and `allowed_callers` unavailable on `mcp_toolset`	Tool safety	D2, D5
`mcp_tool_use` / `mcp_tool_result` blocks	Response handling	D2
Third-party error strings and untrusted results	Reliability and security	D5
`defer_loading` on toolsets and aggregation	Context management	D5

CERTIFICATION NOTE

Exam Tips

- **Beta header** `mcp-client-2025-11-20`**.** The `2025-04-04` version is deprecated and put tool configuration inside the server definition.
- **Only tool calls** of the MCP feature set are supported through the connector.
- `type` **is** `"url"` **only; the URL must be** `https://`**; local STDIO cannot be connected directly.**
- **Every server must be referenced by exactly one toolset,** and every toolset must name a defined server.
- **An unknown tool name in** `configs` **is a warning, not an error.** This is a favourite exam distinction.
- **Precedence:** `configs` **>** `default_config` **> system defaults.** Defaults are `enabled: true`, `defer_loading: false`.
- **Not on Amazon Bedrock or Google Cloud;** on Microsoft Foundry only for Hosted on Anthropic deployments.
- **Not ZDR-eligible.**
- `strict` **and** `allowed_callers` **are available on all tools except** `mcp_toolset`**.**
- **You supply and refresh the OAuth token.**

36.26 Twenty Practice Questions

1. What problem does MCP exist to solve, and what is the arithmetic?
2. Which beta header does the current MCP connector require?
3. Which parts of the MCP specification does the connector support?
4. What two request components does the connector use, and what does each configure?
5. What are the three permitted `type` values for an entry in `mcp_servers`?
6. State the configuration merge precedence.
7. Which of the connector's four validation rules does not produce an error?
8. Why is that rule not an error?
9. What happens to a tool the server adds tomorrow, under an allowlist and under a denylist?
10. Which two tool properties are unavailable on `mcp_toolset`?
11. Name the two response content block types the connector introduces.
12. Which field on an `mcp_tool_use` block tells you which server was called?
13. On which platforms is the MCP connector unavailable?
14. Is the MCP connector covered by zero data retention?
15. Can you connect a local STDIO server through the connector?
16. Where do you set `defer_loading` for MCP tools?
17. Who runs the OAuth flow for an authenticated MCP server?
18. Can MCP connector tools be called from within code execution?
19. Your denylisted tool got called and no error was returned. What happened?
20. A server upgrade tripled your input bill overnight. Why, and what is the fix?

36.27 Answers

1. The *N* × *M* integration problem. *N* applications times *M* systems becomes *N* + *M* with a protocol in the middle. *Distractor:* "to make tools faster" — MCP adds a hop; it removes duplicated work.

2. `mcp-client-2025-11-20`**.** *Distractor:* `mcp-client-2025-04-04`, which is deprecated and is the version where `tool_configuration` lived in the server definition.

3. Tool calls only. *Distractors:* resources, prompts, sampling — all exist in the specification and none is supported through the connector. The client-side SDK helpers reach resources and prompts.

4. `mcp_servers` **configures the connection** (url, name, token); **an** `mcp_toolset` **entry in** `tools` **configures which tools are enabled and how.**

5. One: `"url"`**.** The question is a trap — the documentation says *"Currently only 'url' is supported."*

6. `configs` **>** `default_config` **> system defaults,** where the system defaults are `enabled: true` and `defer_loading: false`.

7. Unknown tool names in `configs`**.** *"A backend warning is logged but no error is returned."*

8. Because MCP servers may have dynamic tool availability. A strict check would break configurations that are legitimately ahead of or behind the server's current list.

9. Under an allowlist it is disabled (`default_config.enabled: false` applies to it); **under a denylist it is enabled** (the default is `true` and your `configs` does not name it). §36.17 measures 3 against 31.

10. `strict` **and** `allowed_callers`**.** Both are documented as available on *"All tools except* `mcp_toolset`*"*.

11. `mcp_tool_use` **and** `mcp_tool_result`**.** *Distractor:* `tool_use`/`tool_result`, which are the ordinary client blocks; *distractor:* `server_tool_use`, which is Anthropic's own server tools.

12. `server_name`**.** With three servers each exposing a `search`, it is the only discriminator.

13. Amazon Bedrock and Google Cloud. On Microsoft Foundry it requires a Hosted on Anthropic deployment.

14. No. *"Data exchanged with MCP servers, including tool definitions and execution results, is retained according to Anthropic's standard data retention policy."*

15. No. The server must be publicly exposed over HTTPS, using Streamable HTTP or SSE. Use the client-side helpers for local servers, or a tunnel.

16. On the toolset — `default_config` **for the whole server, or per tool in** `configs`**.** Not on individual tool definitions, which you did not write.

17. You do. *"API consumers are expected to handle the OAuth flow and obtain the access token prior to making the API call, and to refresh the token as needed."*

18. No. MCP connector tools are listed among those that cannot currently be called programmatically.

19. The `configs` **key did not match a tool name on the server,** so the API logged a backend warning and returned no error. Assert the keys against a fetched tool list, and prefer an allowlist.

20. The server's tool definitions are in your cached prefix, and adding a tool changed the prefix, invalidating the cache. The fix is `defer_loading: true` on the toolset, because deferred tools are excluded before the cache key is computed.

36.28 Ten Practical Exercises

1. Add a fourth server to `mcpcat.py` and re-run every audit. Predict the collision count first.
2. Extend `constraint_audit` to report the worst offender per server rather than a total.
3. Write the `inputSchema` → `input_schema` adapter and a test with a real manifest fixture.
4. Add a `strict`-availability field to `tooldef.Tool` and make the audit read it rather than hard-coding zero.
5. Extend `effect_tools` to take a documented effect list per server and report the difference against the name-based guess.
6. Score all 55 tools' descriptions individually and rank the ten most likely to cause a tie.
7. Compute the cache invalidation cost of one server adding one tool, using `cacheplan`.
8. Build the allowlist your own assistant would need for one real server, and count what it leaves enabled.
9. Add a test asserting that no enabled tool's name matches `EFFECT_CUES` unless it appears in an explicit approvals list.
10. Write the migration from `mcp-client-2025-04-04` to the current shape, as a function with tests.

36.29 Five Challenge Labs

1. **The wrapper generator.** Given an MCP manifest, generate route-(b) wrapper tools with Chapter 34's five gates and Chapter 33-compliant descriptions. Report how many of the 455 linter errors your generated descriptions remove.
2. **A manifest lockfile.** Design and implement a lockfile for MCP servers: pinned tool names, schema hashes, and a diff that fails a build when the server changes. State what it cannot pin.
3. **Measure the collision.** Adapt Chapter 33's paired proxy harness to the three `search` tools and report ties, margins and an exact test — with the proxy boundary stated in the claim, not just the docstring.
4. **Cache invalidation, measured.** Instrument a connector-mounted server, record `cache_read_input_tokens` across a week, and correlate drops with the server's release history. Publish the cost of one upgrade.
5. **A policy gateway.** Build a local MCP server that proxies a remote one, applying an allowlist, call-site validation, idempotency and instructive error strings. Report what it costs in latency and what it buys in Chapter 34's four metrics.

36.30 Ten Interview Questions

1. Why does MCP exist, and what would you have built instead?

2. What does the MCP connector give you, and what does it take away?
3. Allowlist or denylist? Defend your answer with what happens next release.
4. Which of the connector's validation rules does not raise, and why does that matter?
5. A third-party tool's schema says `maximum: 500000`. What enforces it?
6. Where would you put Chapter 34's gates for a connector-mounted tool?
7. When would you run your own MCP client instead of using the connector?
8. What happens to your prompt cache when a connected server ships a new tool?
9. Three servers each expose a `search`. What do you do?
10. What in a connected server's manifest would make you refuse to connect it?

36.31 Summary

Part 7 opens by handing away everything Part 6 taught you to hold. The Model Context Protocol exists because N applications and M systems produce $N \times M$ integrations without a standard in the middle and $N + M$ with one, and that arithmetic is not in dispute: an organisation that has written the same Jira wrapper four times does not need persuading. What MCP standardises is discovery, invocation and capability negotiation — the plumbing. What it does not standardise is authorisation, idempotency, validation or what an error string should say, which is to say that every argument in Chapters 32 to 35 was about the water and none of it has been settled.

The Claude platform offers three routes to a server, and choosing between them is most of the engineering. The **MCP connector** is one JSON object: name a public HTTPS server in `mcp_servers`, configure its tools with an `mcp_toolset` entry in `tools`, and Anthropic's own client does the rest, returning `mcp_tool_use` and `mcp_tool_result` blocks for calls that have already happened. It supports tool calls and nothing else of the specification, it is unavailable on Amazon Bedrock and Google Cloud, and it is not covered by zero data retention. The **client-side SDK helpers** are more code and reach local servers, resources and prompts — and, more importantly, put every gate from Chapter 34 back in your process. **MCP tunnels** make a private server reachable without making it public, and Chapter 38 takes them properly.

The audit is the part worth carrying. Three plausible servers, 55 tools: their schemas declare **277 constraints**, of which **211** would be enforced under `strict: true` and **zero** are, because `strict` is documented as available on *"All tools except* `mcp_toolset`*"*. Sixty-six of the unenforced ones are the range checks Chapter 32 identified as the dangerous misses. The 55 tools carry only **52 distinct names** — three `search` tools and two `create_issue` tools that you cannot rename — and Chapter 33's linter reports **455 errors** across descriptions you cannot edit. Chapter 34's scorer finds **none of the three servers' error strings actionable**, and one of them puts an internal hostname into your conversation on every turn thereafter. Chapter 35's arithmetic is the one that survives intact and is immediately useful: 25,300 definition tokens, above the documented 30-to-50-tool accuracy threshold and past the deferral crossover, so `defer_loading` on the toolset is both cheaper and more accurate.

Two operational decisions follow, and neither is about MCP. The first is **allowlist over denylist**, because the two patterns differ on exactly one question — what happens to a tool the server adds next week — and the answer is 3 against 31. The fourth validation rule sharpens it: an unknown tool name in `configs` is a **backend warning, not an error**, so a misspelled denylist protects nothing and says nothing. The

second is **route the reads through the connector and wrap the writes yourself**, because for a connector-mounted tool there is no call site of yours at which to authorise, validate, de-duplicate or explain, and those four gates were the whole of Chapter 34. Chapters 37 to 40 build the protocol properly — hosts and clients, transports and lifecycle, the primitives beyond tools, and what it takes to ship a server someone else will audit the way this chapter just audited three.

KEY TAKEAWAYS

1. **MCP turns *N* × *M* integrations into *N* + *M*.** That is the whole economic case and it is a good one.
2. **The protocol standardises plumbing, not policy.** Authorisation, idempotency, validation and error quality remain yours.
3. **The connector supports tool calls only,** needs a public HTTPS server, is unavailable on Bedrock and Google Cloud, and is not ZDR-eligible.
4. `strict` **and** `allowed_callers` **are unavailable on** `mcp_toolset` — 277 declared constraints, 0 enforced.
5. **You cannot rename a colliding tool or edit a bad description.** You can disable the tool and disambiguate in the system prompt.
6. **The error string is the server's,** and none of the three audited here was actionable while one leaked an internal hostname.
7. **Allowlist over denylist,** because the two differ only on the tool that does not exist yet.
8. **An unknown tool name in** `configs` **is a warning, not an error.** Assert the keys yourself.
9. `defer_loading` **goes on the toolset,** and it also protects your cache from the server's release schedule.
10. **The trust boundary did not move.** Route reads through the connector and wrap anything with an effect.

36.32 Glossary

Term	Meaning
Allowlist	`default_config.enabled: false` plus specific tools enabled; tomorrow's tool is disabled
`authorization_token`	Optional OAuth bearer on a server definition; you obtain and refresh it
Capability negotiation	Client and server agreeing at connection time on what each supports
`configs`	Per-tool configuration overrides on a toolset, keyed by tool name
`default_config`	Toolset-level defaults, overridden by `configs`
Denylist	Defaults left alone plus specific tools disabled; tomorrow's tool is enabled
MCP	Model Context Protocol; an open standard for connecting AI applications to external systems
MCP connector	Anthropic's server-side MCP client in the Messages API; beta `mcp-client-2025-11-20`

Term	Meaning
`mcp_servers`	Request field defining server connections: `type`, `url`, `name`, `authorization_token`
`mcp_toolset`	The `tools` entry configuring one server's tools; accepts neither `strict` nor `allowed_callers`
`mcp_tool_result`	Response block carrying the server's result and an `is_error` flag
`mcp_tool_use`	Response block for an MCP call, carrying `server_name`
$N \times M$ problem	Writing every application-to-system integration separately
Streamable HTTP / SSE	The two transports the connector accepts; STDIO is not connectable directly

36.33 Further Reading

Primary sources, verified 4 August 2026 at `platform.claude.com/docs/en`:

- `agents-and-tools/mcp-connector` — the request shape, both beta headers and the migration between them, the field tables, merge precedence, the four validation rules, the two block types, the limitations, platform availability, the ZDR statement, batch support and the client-side SDK helpers.
- `agents-and-tools/tool-use/tool-reference` — the optional tool properties table, which is where `strict` and `allowed_callers` are documented as available on *"All tools except* `mcp_toolset`*"*.
- `agents-and-tools/tool-use/tool-search-tool` — the MCP integration section, which is where `defer_loading` for toolsets is specified, and the aggregation case for 200+ tools.
- `agents-and-tools/tool-use/programmatic-tool-calling` — the tool restrictions section listing MCP connector tools as not currently callable from code.
- `agents-and-tools/remote-mcp-servers` — the directory of remote servers, to be used with Chapter 40's review checklist.
- The MCP specification at `modelcontextprotocol.io`, revision **2025-11-25**, in particular the authorization section the connector page links for the OAuth flow.

Where to go next in this book. Chapter 37 separates hosts, clients and servers, which is the distinction this chapter deliberately blurred. **Chapter 38** covers transports and the protocol lifecycle, including tunnels and the STDIO case. **Chapter 39** covers resources and prompts — the primitives the connector does not reach — and the SDK helpers that do. **Chapter 40** turns the audit around: how to build, secure and ship a server that survives being audited this way. **Chapter 41** meets MCP again inside Claude Code. **Chapter 58** takes indirect prompt injection through a third-party tool result, and **Chapter 61** covers governing an approved-server list.